Group 04

MEDI-CINE

Software Development Plan (Small Project)

Version 2.0

Revision History

Date	Version	Description	Author
03/Oct/26	0.1	Initial project proposal and scope baseline	Group 04
13/Oct/26	1.0	Strengthened estimates, schedule controls, deliverable acceptance, and reporting metrics	Group 04
23/Oct/26	2.0	Expanded estimation approach (basis, re-estimation, and cost baseline), sprint planning, scheduling, and resource allocation; enhanced reporting metrics, risk mitigation, and configuration control; added project close-out activities.	Group 04

Table of Contents

1. Introduction 4

1.1 Purpose 4

1.2 Scope 4

1.3 Overview 4

2. Project Overview 5

2.1 Project Purpose, Scope, and Objectives 5

2.2 Assumptions and Constraints 5

2.3 Project Deliverables 6

3. Project Organization 8

3.1 Organizational Structure 8

3.2 Roles and Responsibilities 10

4. Management Process 12

4.1 Project Estimates 12

4.2 Project Plan 12

4.2.1 Phase Plan 12
4.2.2 Iteration Objectives 14
4.2.3 Releases 15
4.2.4 Project Schedule 16
4.2.5 Project Resourcing 18

4.3 Project Monitoring and Control 20

4.3.1 Requirements Management 20
4.3.2 Reporting and Measurement 21
4.3.3 Risk Management 22
4.3.4 Configuration Management 23

4.4 Project Close-out 24

Software Development Plan (Small Project)

1. Introduction

This *Software Development Plan* (SDP) introduces the planning and organization of the MEDI-CINE project. It provides a framework for guiding the development of the system and serves as a reference for the project team throughout the software development lifecycle. The plan is based on the *MEDI-CINE Vision Document* and is organized into sections covering project organization, management, scheduling, monitoring, risk management, and configuration management.

1.1 Purpose

The purpose of this *Software Development Plan* is to establish a common basis for coordinating project activities, defining responsibilities, and ensuring that the MEDI-CINE system is developed according to its specified requirements. It helps the project team organize their work, track project deliverables, and maintain consistency throughout the development process.

The following people use the *Software Development Plan*:

- **The team leader / project manager** uses it to plan the project schedule, identify resource requirements, assign and coordinate tasks, manage project risks, and monitor overall progress against project objectives.
- **The secretary** uses it to maintain project documentation, record meeting minutes and progress updates, and keep project information organized and up to date.
- **Project team members** use it to understand their assigned tasks, responsibilities, deadlines, technical requirements, and dependencies on other activities. Developers use it to coordinate design, implementation, and integration, while testers use it to plan testing activities, verify system functionality, and report defects.

1.2 Scope

This Software Development Plan (SDP) applies to the MEDI-CINE project, a web-based cinema management and online ticket-booking platform. It covers the development and deployment of two main components: a customer-facing web application and an administration/staff dashboard used by cinema staff and administrators.

The customer-facing application provides features for browsing movies, viewing showtimes, selecting seats, booking tickets, making payments, and accessing electronic tickets and booking history, submitting movie reviews and ratings, and receiving notifications and reminders. The administration/staff dashboard supports movie management, screening room and seat management, showtime scheduling, ticket pricing, promotion management, operational reporting, and QR-code ticket validation for admission control.

The scope of this plan includes the major software development activities from requirements analysis and UI/UX design to implementation, integration, testing, deployment, and project close-out (handover of deliverables and documentation). The plans outlined in this document are based on the product requirements defined in the MEDI-CINE Vision Document. Detailed tasks, schedules, and deliverables for individual iterations will be described in the corresponding Iteration Plans.

PWA (Progressive Web App) support and the AI-assisted movie recommendation feature are considered optional extensions. Their implementation will depend on the available time and resources and must not compromise the delivery of the core system functionality.

1.3 Overview

This Software Development Plan is organized into the following sections:

- Project Overview — describes the project's purpose, scope, objectives, assumptions, constraints, and expected deliverables.
- Project Organization — describes the project's organizational structure and defines the roles and responsibilities of team members.
- Management Process (Section 4) — describes project estimates, development phases, iteration objectives, releases, project scheduling, resource allocation, and project close-out (Section 4.4); it also contains the monitoring and control subsections below.
- Project Monitoring and Control (Section 4.3) — describes how requirements, project progress, performance, and risks are monitored and controlled.
- Configuration Management (Section 4.3.4, within Project Monitoring and Control) — describes how project artifacts, software versions, changes, and baselines are managed throughout the development lifecycle.

2. Project Overview

2.1 Project Purpose, Scope, and Objectives

Project Purpose

The MEDI-CINE project aims to address fragmented cinema operations and inconvenient ticket-booking processes by providing a unified platform that connects customer booking, cinema administration, payment processing, and ticket admission. The system is intended to improve the customer experience, maintain consistent operational data, and reduce manual work for cinema staff.

Project Scope

The project covers the development of a web-based cinema management and online ticket-booking system consisting of a customer-facing web application and an administration/staff dashboard. The customer application supports movie discovery, showtime browsing, seat selection, booking, payment, booking history, electronic ticket access, movie reviews and ratings, and notifications and reminders. The administration/staff dashboard supports user and cinema data management, including movies, screening rooms, seats, showtimes, ticket pricing, promotions, and operational reports. The system also includes QR-code ticket validation to support admission control. The core release comprises the 14 capabilities defined in the MEDI-CINE Vision Document v1.2. AI-assisted movie/showtime recommendations and PWA support are optional extensions and are not required for completion of the core system.

Project Objectives

The project objectives are to:

- Provide customers with a convenient and accessible online movie-ticket booking experience.
- Maintain correct seat availability and prevent duplicate bookings during concurrent booking attempts.
- Ensure that electronic tickets are issued only after payment confirmation and that each ticket can be used for admission only once.
- Enable cinema staff to validate tickets efficiently and administrators to maintain consistent cinema data.
- Deliver a functional, secure, and tested system with documented setup, deployment, and testing procedures.

2.2 Assumptions and Constraints

Assumptions

- The project is developed as an academic group project, and all team members are expected to remain available at a level sufficient to perform their assigned development, testing,

documentation, and coordination tasks throughout the project schedule; reduced availability is managed as a risk (Section 4.3.3).

- The target environment represents a medium-sized cinema operation with approximately 15–20 screening rooms, as described in the MEDI-CINE Vision Document v1.2.
- Customers, cinema staff, and administrators have access to supported web browsers and a network connection to use the system.
- A payment sandbox or simulated payment gateway will be available for development and testing. Integration with a live payment provider is not assumed to be mandatory.
- The team can use suitable development tools, database technology, version control, and testing tools to implement and evaluate the system.
- Requirements and acceptance criteria will be refined with reference to the MEDI-CINE Vision Document v1.2, course requirements, and feedback from the project supervisor.
- AI-assisted movie/showtime recommendations and Progressive Web App (PWA) support are optional and will be considered only if the core system is sufficiently complete and stable.

Constraints

- Schedule: The project must follow the agreed academic schedule and development roadmap. The roadmap consists of Phase 1 – Inception (Sprint 1), Phase 2 – Elaboration (Sprints 2–3), and Phase 3 – Construction (Sprints 4–5).
- Human resources: The project is limited to the existing five-member team. Tasks must be allocated according to team members' responsibilities and available time.
- Project scope: The release baseline consists of the 14 core capabilities defined in the MEDI-CINE Vision Document v1.2. Optional AI and PWA features must not compromise the completion or quality of core functionality.
- Technology and platform: MEDI-CINE is a web-based application with a responsive customer interface and an administration/staff dashboard. The project does not require a native mobile application.
- Payment integration: Payment processing is expected to use a sandbox or simulated gateway for the project demonstration. Production payment processing, financial settlement, and live-provider requirements are outside the assumed baseline unless explicitly approved.
- Data and security: The system must enforce role-based access control and protect relevant account and booking information. Seat availability, booking status, payment confirmation, and ticket status must remain consistent during concurrent operations.
- Quality and performance: The team must test the core booking and admission workflows, including concurrent seat selection, failed or expired payments, QR-ticket validation, and prevention of repeated ticket use. Provisional performance targets in the Vision Document will be validated against the agreed test environment.
- Budget and external services: No specific monetary budget or paid-service allowance has been established in the available project baseline. The implementation should therefore prioritize available academic resources and free or appropriately licensed tools and services.

2.3 Project Deliverables

The project will deliver a requirements baseline, UX flows and approximately 15–20 responsive screens, a working customer web application, an administration/staff dashboard, database and API components, payment integration in a sandbox environment, QR e-ticket generation, test cases and results, deployment

documentation, and a final demonstration package.

The functional baseline includes 14 core capabilities: authentication and authorization; movie catalogue management; cinema room and seat management; showtime scheduling; ticket pricing; movie and showtime discovery; seat selection and temporary holds; booking and payment; QR e-ticket and check-in; booking history; dashboard and reports; reviews and ratings; promotions and discount codes; and notifications and reminders. AI-assisted movie/showtime recommendations and PWA support are optional planned extensions and are excluded from the core release commitment.

The target dates in this table follow the five-sprint schedule defined in Section 4.2.4. Any subsequent changes will be recorded and approved through the agreed schedule change-control process.

No.	Deliverable	Target milestone	Target delivery date	Acceptance criteria
1	Requirements baseline and prioritized backlog	End of Sprint 2	24/Oct/26	Core functional and non-functional requirements, priorities, assumptions, constraints, and acceptance criteria are documented and reviewed by the project team.
2	UX flows and responsive screen designs	End of Sprint 3	07/Nov/26	UX flows for the main customer and administration/staff tasks are documented, with approximately 15–20 responsive screens designed and reviewed.
3	Database design and API contracts	End of Sprint 3	07/Nov/26	The database schema, key entities and relationships, and API contracts for core workflows are documented and reviewed.
4	Initial customer web application and administration/staff dashboard	End of Sprint 4, week 1	14/Nov/26	The main interfaces are implemented, and the core customer and administrative workflows can be demonstrated.
5	End-to-end booking workflow, including seat selection, temporary holds, sandbox payment, QR e-ticket generation, and check-in	End of Sprint 4, week 1	14/Nov/26	A complete booking scenario is demonstrated. Seat availability is checked by the backend, a QR e-ticket is issued only after payment confirmation, and a used ticket is rejected on a subsequent admission attempt.
6	Integrated beta release	End of Sprint 4	21/Nov/26	The customer application, administration/staff dashboard, database, APIs, and sandbox payment flow are integrated. Core workflows are ready for systematic testing, and identified defects are recorded.
7	Test cases, test execution results, and beta test report	First week of Sprint 5	28/Nov/26	Test cases cover core functionality, integration, access control, payment outcomes, seat availability, and QR-ticket validation. Execution results,

				defects, and unresolved issues are documented.
8	Release Candidate (Build 0.9) and Final Test Report	Second week of Sprint 5	05/Dec/26	Completion of the Release Candidate (Build 0.9), final testing, and test report. The release candidate will undergo final verification before the official academic release.
9	Deployment and setup documentation	Second week of Sprint 5	05/Dec/26	Documentation explains the required environment, configuration, setup, database preparation, deployment steps, and relevant limitations. A draft is delivered by 05/Dec/26, validated during the deployment rehearsal (07/Dec/26–10/Dec/26), and finalized by 11/Dec/26.
10	Final demonstration package	End of Sprint 5	12/Dec/26	The package includes the final demonstration materials and supporting evidence showing the main customer, administration/staff, booking, payment, and admission workflows.

Delivery Schedule and Acceptance

Sprint 1 focuses on establishing the project foundation, including the initial scope, draft requirements, backlog, and team responsibilities. Sprint 2 finalizes the requirements baseline and prioritized backlog and implements initial administration capabilities. Sprint 3 completes the UX flows, responsive screen designs, database design, and API contracts. Sprint 4 delivers the initial customer application and administration/staff dashboard, demonstrates the end-to-end booking workflow, and produces the integrated beta release. Sprint 5 focuses on systematic testing, defect resolution, final verification, deployment documentation, and the final demonstration package.

Each deliverable will be reviewed against its stated acceptance criteria. Any outstanding defects or limitations will be documented and assessed before final acceptance. The target dates are defined in the current project schedule. Any subsequent changes will be recorded through the agreed schedule change-control process.

The final release must include all 14 core capabilities defined in the MEDI-CINE Vision Document v1.2. AI-assisted movie/showtime recommendations and PWA support remain optional extensions and are not required for acceptance of the core release.

3. Project Organization

3.1 Organizational Structure

The MEDI-CINE project is organized as a five-member student development team. The Team Leader / Project Manager coordinates project activities, assigns tasks, monitors progress, and facilitates communication among team members. The remaining members contribute to frontend development, backend development, data engineering, AI-related exploration, testing, documentation, and deployment according to their assigned roles.

The course instructor or project supervisor (referred to below as the project supervisor) serves as the external review authority, providing academic guidance and evaluating project deliverables against course requirements, where applicable.

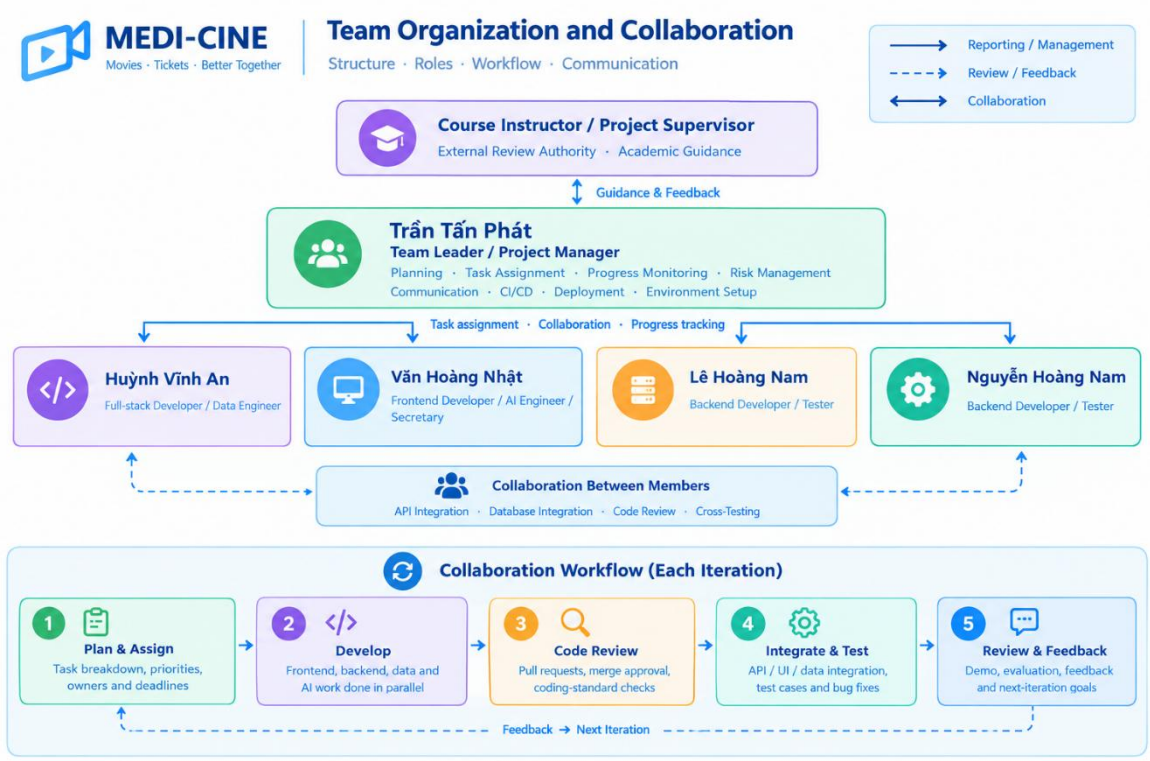
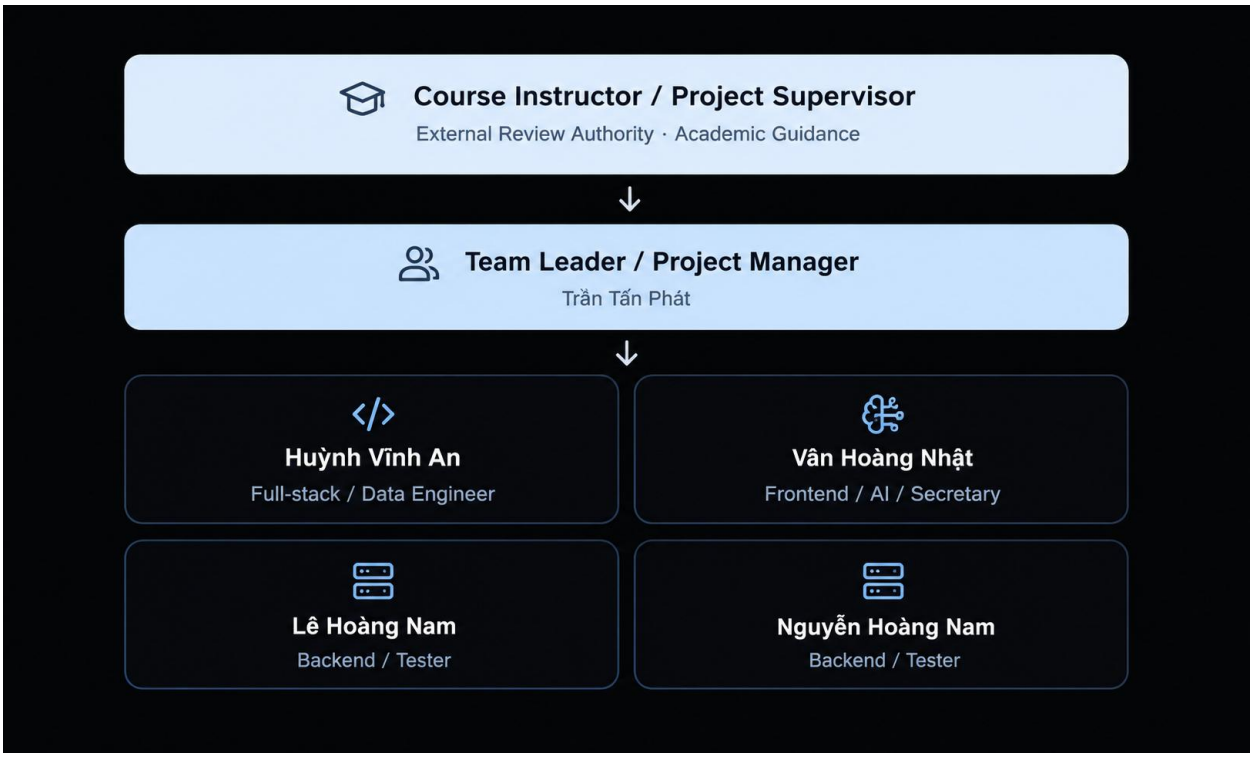


Figure 1. MEDI-CINE Team Organizational Structure and Collaboration Workflow



The team follows an iterative collaboration workflow consisting of planning and task assignment, development, code review, integration and testing, and review and feedback. Team members coordinate through technical discussions, code reviews, API and database integration, cross-testing, and defect resolution. Feedback from internal reviews and the project supervisor is used to guide subsequent iterations.

3.2 Roles and Responsibilities

The following table defines the responsibilities of each project participant. Responsibilities are assigned according to the team's technical roles and supporting functions to ensure that project management, software development, quality assurance, documentation, and deployment activities are covered.

Person	**Role**
Course Instructor / Project Supervisor	**Academic Guidance and Project Review:** Provides academic guidance throughout the project lifecycle, reviews project plans and deliverables, evaluates the team's progress against course requirements, and provides feedback for improvement.
Trần Tấn Phát, Team Leader / Project Manager	**Project Planning and Coordination:** Defines project milestones, organizes tasks, assigns responsibilities, monitors progress, and coordinates communication among team members. **Risk and Quality Management:** Identifies project risks, tracks issues, coordinates problem resolution, and monitors overall deliverable quality. **Full-stack Development and Deployment:** Contributes to frontend and backend implementation, system integration, and application deployment as needed.

Huỳnh Vĩnh An, Full-stack Developer / Data Engineer	**Application Development:** Implements assigned frontend and backend features and supports integration between application components. **Database Design and Management:** Designs and maintains database schemas, relationships, constraints, and data-access operations. **Data Integrity**: Ensures data consistency and supports database debugging, optimization, and testing.
Vân Hoàng Nhật, Frontend Developer / AI Engineer / Secretary	**Frontend Development:** Designs and implements responsive user interfaces, reusable UI components, and customer-facing interactions for the MEDI-CINE platform. **AI Feature Exploration:** Researches and prototypes optional AI-powered features where feasible within the project scope. **Documentation and Administration**: Records meeting minutes, maintains project documentation, tracks decisions, and supports the preparation of reports and presentation materials.
Lê Hoàng Nam, Backend Developer / Tester	**Backend Development:** Implements server-side business logic, REST APIs, request validation, and interactions with the database. **System Integration:** Integrates backend services with frontend components and other system modules. **Software Testing:** Designs and executes functional and integration test cases, identifies and reports defects, verifies fixes, and supports regression testing.
Nguyễn Hoàng Nam, Backend Developer / Tester	**Backend Development**: Implements and maintains assigned backend services, API endpoints, and business logic. **Feature Integration:** Supports integration among booking, seat management, payment, and ticket-related modules. **Workflow Testing:** Tests critical workflows, including seat availability, booking confirmation, payment processing, and QR-code ticket validation; documents defects and verifies corrective changes.

Shared Processes and Supporting Responsibilities

In addition to their assigned responsibilities, all team members contribute to the following supporting processes:

- Requirements and change management: Team members clarify requirements, identify proposed changes, and communicate their impact on scope and schedule. The Team Leader coordinates the review and prioritization of changes with the team.
- Configuration management and code review: Developers follow the agreed repository and version-control practices, review code changes, and resolve integration conflicts before changes are incorporated into the shared codebase.
- Integration and quality assurance: Developers collaborate to integrate frontend, backend, database, and other components. Team members participate in testing, defect reporting, regression testing, and verification of fixes.
- Documentation and communication: The Secretary maintains meeting minutes and project records, while all members document relevant technical decisions, implementation details, and testing results.

- Progress monitoring and delivery: The Team Leader tracks tasks, milestones, risks, and dependencies. Each responsible member provides progress updates and prepares assigned deliverables for internal review.
- Review and feedback: The team reviews deliverables against documented requirements and acceptance criteria. Feedback from internal reviews and the project supervisor is considered when planning subsequent iterations.

All team members share responsibility for product quality and successful integration. The Team Leader coordinates these activities, while the project supervisor provides external academic guidance and evaluation rather than participating in the team's day-to-day development tasks.

4. Management Process

4.1 Project Estimates

The initial project estimate is based on the agreed scope of 14 core capabilities, a five-member student team, three development phases, and five sprints. It covers requirements analysis, UX design, implementation, integration, testing, deployment preparation, and documentation.

Schedule Estimate: The project follows the current five-sprint schedule documented in Section 4.2.4. Planned start dates, end dates, review dates, and milestone targets are maintained in the sprint calendar. Any subsequent changes will be reviewed and documented through the agreed schedule change-control process.

Effort Estimate: Before each sprint, team members estimate the effort required for their assigned backlog items in hours or person-days. The Team Leader consolidates these estimates and checks them against team members' availability, task dependencies, and the remaining academic schedule. Total effort estimates are updated as the backlog is refined and are derived from task-level estimates rather than unsupported fixed values.

Cost Estimate: The project is developed by a five-member student team. The team prioritizes free or student-accessible tools and a payment sandbox, where available. Any expected expenses for hosting, domain registration, or third-party services must be recorded in Vietnamese dong (VND) before the budget baseline is approved. If no paid services are used, the planned cash expenditure is VND 0; any later expense is recorded and approved as a change to the cost baseline. This estimate excludes the imputed value of team members' development effort.

Estimation Basis and Re-estimation: Estimates are based on the Vision Document, the prioritized product backlog, the team's available skills and capacity, technical dependencies, and the academic calendar. Effort, schedule, and cost estimates are reviewed at the end of each sprint, whenever a major technical dependency changes, or when scope changes or significant defects affect planned work. The Team Leader coordinates each review, records revised estimates and the reasons for the changes, and updates the project baseline after the team agrees on the revisions.

4.2 Project Plan

MEDI-CINE is organized into three development phases and five sprints. Each sprint is expected to produce a demonstrable increment of the software. Development priorities focus first on establishing the project foundation, then implementing the cinema administration and customer booking workflows, and finally integrating, testing, and preparing the final demonstration build.

The 14 core capabilities form the baseline scope. The AI movie/showtime assistant and Progressive Web App (PWA) support are optional extensions and will only be considered when the core booking workflow is sufficiently stable and the remaining schedule permits.

4.2.1 Phase Plan

Because MEDI-CINE is a small student project, a separate formal Work Breakdown Structure (WBS) is

not required. The product backlog and sprint task lists will serve as the primary work breakdown mechanism. Each task will include an owner, an estimate, dependencies, and acceptance criteria.

Development Phases

Phase	Iterations	Main Activities	Milestones and Achievement Criteria
Inception	Sprint 1	Confirm the problem statement, target users, project scope, initial requirements and backlog, team responsibilities, repository, and initial architecture.	Initial project scope, requirements, and backlog reviewed and agreed upon by the team; project repository established; team responsibilities and initial architecture documented.
Elaboration	Sprints 2–3	Finalize the requirements baseline and prioritized backlog. Refine user experience flows, responsive interface designs, database model, API contracts, technical architecture, risk assessment, and test approach. Begin developing administration and booking workflows.	Requirements baseline and prioritized backlog completed by the end of Sprint 2. UX flows, responsive interface designs, database design, and API contracts reviewed by the end of Sprint 3. Initial administration capabilities demonstrated.
Construction	Sprints 4–5	Deliver the initial customer application and administration/staff dashboard; demonstrate the end-to-end booking workflow; integrate core capabilities; conduct functional, integration, and regression testing; correct defects; prepare deployment and complete project documentation.	End-to-end booking demonstration completed by 14 November 2026. Integrated beta build available by 21 November 2026. Beta test report completed by 28 November 2026. Release Candidate (Build 0.9) and draft deployment/setup documentation completed by 5 December 2026; deployment/setup documentation finalized after the deployment rehearsal by 11 December 2026. Final demonstration package completed by 12 December 2026.

Timeline and Release Points

Phase / Sprint	Planned Timing	Main Deliverable	Review Point
Inception – Sprint 1	12/Oct/26 – 17/Oct/26	Initial project scope, draft requirements and backlog, team responsibilities, initial architecture, authentication foundation, basic navigation, and Build 0.1	Project setup and initial scope review
Elaboration – Sprint 2	19/Oct/26 – 24/Oct/26	Requirements baseline, prioritized backlog, initial administration	Requirements baseline review

		capabilities, and Build 0.2	
Elaboration – Sprint 3	26/Oct/26 – 07/Nov/26	UX flows, responsive interface designs, database design, API contracts, initial customer booking workflow prototype, and Build 0.3	UX, database, and API review
Construction – Sprint 4	09/Nov/26 – 21/Nov/26	Initial customer application and administration/staff dashboard; end-to-end booking demonstration; integrated beta (Build 0.4)	Booking demonstration; integrated beta review
Construction – Sprint 5	23/Nov/26 – 12/Dec/26	Beta test report, Release Candidate (Build 0.9), deployment/setup documentation, and Build 1.0	Beta test report; final acceptance review and academic demonstration

The sprint dates above have been confirmed against the academic calendar; any change follows the schedule change-control process. At each review point, the team will demonstrate the available build, check the sprint objectives, review test evidence and outstanding defects, and decide whether the relevant milestone has been achieved.

4.2.2 Iteration Objectives

Each sprint has a defined objective and a demonstrable output. Sprint completion will be assessed against the planned deliverables and their acceptance criteria.

Iteration	**Objectives**	**Expected Deliverables**
Sprint 1 – Inception (12/Oct/26 – 17/Oct/26)	Establish the project foundation and agree on the initial scope, requirements, team responsibilities, and development approach.	Draft requirements and initial backlog reviewed by the team, repository, initial architecture, data model outline, authentication foundation, basic navigation, and Build 0.1.
Sprint 2 – Elaboration (19/Oct/26 – 24/Oct/26)	Refine the architecture and implement the first administration and cinema data management capabilities.	Requirements baseline and prioritized backlog; initial database structure and draft API contracts; initial movie, cinema room, seat, and showtime management capabilities; Build 0.2.
Sprint 3 – Elaboration (26/Oct/26 – 07/Nov/26)	Complete the UX flows, responsive screen designs, database design, and API contracts, while developing an initial customer booking workflow for internal review.	Reviewed UX flows and responsive screen designs; finalized database design and API contracts; an initial customer booking workflow prototype for internal review; Build 0.3.
Sprint 4 –	Integrate and validate the core	Initial customer web application and

Construction (09/Nov/26 – 21/Nov/26)	customer booking and administration/staff workflows, deliver the end-to-end booking demonstration, and prepare the integrated beta release. Prioritize the booking, payment, and admission workflows; if needed, move lower-priority core capabilities (reviews and ratings, promotions and discount codes, notifications and reminders) to Sprint 5 to protect integration quality and the beta milestone.	administration/staff dashboard; end-to-end booking demonstration (14/Nov/26); integrated beta (Build 0.4, 21/Nov/26). Lower-priority core capabilities may move to Sprint 5 but must be completed and tested before the Release Candidate (05/Dec/26).
Sprint 5 – Construction (23/Nov/26 – 12/Dec/26)	Stabilize the software, verify core requirements, prepare the final demonstration, and complete project documentation.	Release Candidate (Build 0.9) with corrected defects, final test evidence, deployment rehearsal (07/Dec/26– 10/Dec/26), completed documentation, and Build 1.0.

Implementation in Sprints 2 and 3 is based on the draft database structure, draft API contracts, and working UX flows; the versions finalized and reviewed on 07/Nov/26 are reconciled with the implemented work at the start of Sprint 4, and any resulting rework is planned as backlog items. AI assistant and PWA capabilities are not mandatory sprint objectives. They may be included only if the 14 core capabilities remain on schedule and sufficient time is available for testing.

4.2.3 Releases

MEDI-CINE uses incremental internal builds to demonstrate progress and obtain feedback. The build numbers below are project planning labels and do not imply that every build is a public production release.

Release	Type	Description and Exit Criteria
Build 0.1 (17/Oct/26)	Internal prototype	Demonstrates the project foundation, initial navigation, authentication foundation, and initial architecture. Requirements and setup deliverables must be available for review.
Build 0.2 (24/Oct/26)	Feature prototype	Demonstrates the first administration capabilities for movies, rooms, seats, and showtimes. The implemented features must be reviewable and their principal operations tested.
Build 0.3 (07/Nov/26)	Functional demonstration	Demonstrates the initial customer booking workflow, including movie and showtime browsing, seat selection, booking creation, and sandbox payment integration or a clearly labelled simulated payment flow. The initial workflow is available for internal review, with the

		complete end-to-end demonstration scheduled for 14/Nov/26.
Build 0.4 (21/Nov/26)	Integrated beta	Combines the planned core capabilities into an integrated build. The team performs integration and regression testing, records known defects, and assesses readiness for final stabilization.
Build 0.9 (05/Dec/26)	Release candidate	Corrected build covering the 14 core capabilities, produced after beta defect correction. The team performs final verification and regression testing and completes the final test report and draft deployment/setup documentation. After this build, only critical defect fixes are accepted before Build 1.0.
Build 1.0 (12/Dec/26)	Final academic release	Represents the final project build for evaluation and demonstration. Core requirements must be verified against acceptance criteria, critical defects must be resolved or explicitly documented, and final documentation must be available.

Each release will be tagged in the version control repository. The team will record the included features, test status, known limitations, and relevant documentation. A release will not be considered complete solely because its planned end date has been reached; its deliverables and review evidence must also be available.

4.2.4 Project Schedule

The project schedule will be maintained as a sprint calendar or Gantt chart. It includes planned start and end dates, review dates, task owners, dependencies, milestones, and completion status. The project is organized into five sprints running from 12 October 2026 to 12 December 2026. The following table summarizes the planned sprint schedule, while the subsequent table identifies key schedule items, target completion points, completion criteria, and responsible roles.

Sprint	**Start Date**	**End Date**	**Duration**
Sprint 1	12/Oct/26	17/Oct/26	6 days
Sprint 2	19/Oct/26	24/Oct/26	6 days
Sprint 3	26/Oct/26	07/Nov/26	13 days
Sprint 4	09/Nov/26	21/Nov/26	13 days
Sprint 5	23/Nov/26	12/Dec/26	20 days

Schedule Item	Target Completion	Completion Criteria	Responsible Role
Requirements and backlog baseline	End of Sprint 2 — 24/Oct/26	Initial scope, core requirements, backlog, and team responsibilities reviewed and documented.	Team Leader, all team members
UX, database, and API review	End of Sprint 3 — 07/Nov/26	UX flows, finalized database design, and API contracts reviewed and baselined by the team. Draft versions used in Sprints 2–3 are reconciled with the implemented work at the start of Sprint 4.	Team Leader, Data Engineer, Backend and Frontend members
End-to-end booking demonstration	End of Sprint 4, week 1 — 14/Nov/26	Movie browsing, seat selection with temporary hold, booking, sandbox payment, QR e-ticket issuance after payment confirmation, and check-in (including rejection of an already-used ticket) demonstrated using a reproducible test scenario.	Backend, Frontend, and Testing members
Integrated beta release	End of Sprint 4 — 21/Nov/26	Planned core capabilities are integrated into Build 0.4. The integrated beta is ready for systematic testing. Initial integration results, known defects, and remaining work are documented. Beta testing and preparation of the beta test report are scheduled for completion by 28/Nov/26.	All developers and testers
Release Candidate (Build 0.9) and final testing	05/Dec/26	Release Candidate (Build 0.9) prepared, final testing completed, and final test report documented.	Team Leader and all team members

		Remaining issues and verification results are recorded before the official academic release.	
Final acceptance review and release (Build 1.0)	End of Sprint 5 — 12/Dec/26	Core acceptance criteria reviewed, critical defects addressed or documented, deployment rehearsal and final demonstration completed, and project documents archived. Build 1.0 is frozen and tagged by 11/Dec/26 so that the demonstration on 12/Dec/26 uses a fixed baseline, and is finalized for the academic release.	Team Leader and all team members

Schedule Monitoring and Corrective Action

The Team Leader will review project progress at least weekly and at the end of each sprint. Actual progress will be compared with the sprint baseline. When a task is delayed, its owner will report the cause, the affected dependencies, and the expected impact on the milestone. The team will then decide whether to resolve the dependency, reassign work, adjust task sequencing, or defer lower-priority optional scope.

Any change affecting a milestone or the final delivery will be recorded, together with its reason and the agreed corrective action. Optional features will not take priority over completion and verification of the 14 core capabilities.

4.2.5 Project Resourcing

MEDI-CINE will be developed by the existing five-member student team. Team members will be assigned work according to their primary responsibilities while collaborating across implementation, integration, testing, and documentation.

Staffing Plan

Team Member	**Primary Role**	**Main Responsibilities and Required Skills**
Trần Tấn Phát	Team Leader / Project Manager / Full-stack Developer (Deployment)	Project coordination, task assignment, schedule and risk monitoring, full-stack development, integration, repository management, and deployment preparation.
Huỳnh Vĩnh An	Full-stack Developer / Data Engineer	Database design, data modelling, data management, API and application integration, and full-stack implementation.

Vân Hoàng Nhật	Frontend Developer / AI Engineer / Secretary	Meeting minutes and project records, frontend implementation, user interface integration, and optional AI-related work if approved.
Lê Hoàng Nam	Backend Developer / Tester	Backend development, API implementation, test case preparation, functional testing, defect reporting, and regression testing.
Nguyễn Hoàng Nam	Backend Developer / Tester	Backend development, business logic implementation, integration testing, defect verification, and support for release validation.

The planned staffing level is five student team members throughout the project. Task allocation and individual workload will be reviewed at each sprint planning meeting. No additional staff are assumed in the baseline.

Staff Acquisition

The project will use the five members of the existing student team, so external recruitment is not planned. If a skill gap or workload issue arises, the Team Leader will first consider task reassignment, peer support, self-study, and consultation with the project supervisor. Any significant change to team composition or project responsibilities will be documented and, where required, discussed with the supervisor.

Tools and Other Resources

The team will use the following resources as needed:

- A shared Git repository for source code and version control.
- An issue tracker or shared task board for backlog items, assignments, defects, and progress.
- A development database environment and documented database scripts.
- Test cinema data, including movies, rooms, seats, showtimes, and bookings.
- A payment sandbox or controlled test alternative for payment workflow validation.
- Desktop browsers and available mobile devices for responsive interface testing.
- Shared project documentation and a workspace for reports, diagrams, and test evidence.

Training and Knowledge Sharing

Training or Knowledge-Sharing Activity	Participants	Target Completion
Repository workflow, branching, code review, and issue tracking	All team members	Sprint 1, before the first collaborative implementation tasks
Database conventions and API contracts	Backend, Data Engineer, and Frontend members as relevant	Before Sprint 2 integration work
Frontend/backend integration and error handling	Frontend and Backend members	Sprint 3, before the booking workflow prototype is built for the Sprint 4 end-to-end

		demonstration
Payment sandbox and callback handling	Members responsible for payment integration and testing	Before payment integration testing in Sprint 3
QR ticket validation and booking verification	Backend and Testing members	Before QR ticket and booking verification in Sprint 4
Deployment, release tagging, and recovery procedures	Team Leader and relevant developers	Before the final deployment rehearsal in Sprint 5

The team will use short peer-learning sessions, technical walkthroughs, and shared implementation notes rather than assuming that every member already has all required skills. The Team Leader will record completed training or knowledge-sharing activities in the project workspace.

4.3 Project Monitoring and Control

Project progress will be monitored against the approved scope, sprint objectives, schedule, effort estimates, test results, and release milestones. The Team Leader will coordinate monitoring activities, while each task owner is responsible for reporting progress, identifying blockers, and providing evidence that assigned work meets its acceptance criteria.

The main control indicators are:

- Completion status of backlog items and core capabilities.
- Planned versus actual schedule and effort.
- Number of open and closed defects and their severity.
- Test execution results and acceptance-test pass rate.
- Readiness of the end-to-end booking workflow.
- Outstanding integration dependencies and release blockers.
- Risk status and the effectiveness of mitigation actions.
- Actual project expenses compared with the approved cost baseline, if any.

Deviations will be discussed during weekly progress reviews and sprint reviews. Corrective actions, scope decisions, and changes to estimates will be documented in the issue tracker or project records.

4.3.1 Requirements Management

The requirements for MEDI-CINE are based on the project Vision Document and are maintained in a prioritized product backlog. Each requirement or backlog item should identify the relevant user role, a clear description, priority, acceptance criteria, dependencies, and implementation status.

The 14 core capabilities constitute the baseline scope. The AI movie/showtime assistant and PWA support are optional and will be considered only if they do not jeopardize delivery of the core capabilities.

Requirements Change Control

1. A team member records a proposed requirement change as a backlog item or change request, including its reason and expected benefit.
2. The Team Leader and affected task owners review the request for scope, effort, schedule, cost, dependency, and testing impacts.
3. The team prioritizes the request against existing sprint commitments and the core requirements.
4. The decision is recorded as accepted, deferred, or rejected, with a brief justification.

5. Accepted changes are incorporated into the backlog and, when necessary, the schedule and effort estimates are updated.
6. Implemented changes are verified against their acceptance criteria and demonstrated during a review before being marked complete.

Changes that materially affect the agreed project scope or academic deliverables will be discussed with the project supervisor when approval is required. The team will maintain traceability between the Vision Document, backlog items, implementation tasks, test cases, and release notes.

4.3.2 Reporting and Measurement

Project reports will be generated throughout the five-sprint lifecycle. The team will use the collected information to evaluate progress, identify schedule or quality problems, revise estimates, and support release decisions.

Reporting Plan

Report or Metric	Frequency	Prepared By / Audience	Purpose
Sprint progress and backlog status	Weekly and at the end of each sprint	Task owners; reviewed by the Team Leader and team	Compare completed work with sprint objectives and identify blockers.
Schedule and effort variance	At the end of each sprint and after significant changes	Team Leader, using task-owner estimates	Identify delays, workload imbalance, and the need to revise estimates or task assignments.
Open and closed defects by severity	Weekly and during integration testing	Testing members; reviewed by the team	Track defect trends, prioritize corrections, and assess release readiness.
Test execution and pass rate	During testing and at each sprint review	Testing members; shared with developers and the Team Leader	Measure verification progress and identify untested or failing functionality.
Core capability completion	At each sprint review	Team Leader and feature owners	Demonstrate progress against the 14 core capabilities and identify incomplete requirements.
Risk register and mitigation status	Weekly and at each sprint boundary	Risk owners; coordinated by the Team Leader	Identify changes in likelihood or impact and determine whether mitigation is effective.
Cost and expense summary	When expenses occur and at each sprint end	Team Leader	Compare actual spending with the approved cost assumptions and identify additional

			expenses.
Release readiness summary	Before each major demonstration or release	Team Leader and relevant developers/testers	Summarize included features, test evidence, known defects, and outstanding release risks.

The minimal set of project metrics will include completed tasks, planned versus actual effort, schedule variance, open and closed defects, test cases executed and passed, core capabilities completed, and actual expenses where applicable. Trend information will be used where repeated measurements are available.

Earned value may be used if the team has a sufficiently defined baseline and reliable task-completion data. Otherwise, progress will be measured using backlog completion and planned-versus-actual effort, without presenting unsupported earned-value figures.

Internal reports will be maintained in the shared project workspace and discussed during team meetings. A concise progress summary, significant risks, and major deviations will be communicated to the project supervisor at agreed review points or when a significant issue threatens a milestone.

4.3.3 Risk Management

Risks were identified during Inception (Sprint 1) and are reviewed at least weekly and at the end of every sprint. The team will evaluate each risk according to its likelihood, potential impact on scope, schedule, cost, or quality, and the urgency of the response. Risks with the greatest potential impact will receive the highest priority.

Each risk will have an owner responsible for monitoring warning signs, updating its status, and reporting changes to the Team Leader. Mitigation actions will be incorporated into the backlog or sprint tasks when appropriate. If a risk materializes, the team will apply the contingency plan and reassess its effect on the schedule and release scope.

Risk Ranking (High, Medium, Low)	**Risk Description and Impact**	**Mitigation Strategy and/or Contingency Plan**
High	**Seat conflicts or stale seat availability.** Concurrent users or outdated seat status may result in conflicting bookings or an incorrect seat-selection experience.	Implement server-side seat availability validation, temporary seat holds with expiry, and transactional updates where appropriate. Test concurrent booking scenarios and verify that expired holds are released. If a conflict occurs, reject or recover the affected booking safely and refresh seat availability.
High	**Payment sandbox or callback problems.** Provider configuration, callback failures, or sandbox instability may prevent the team from demonstrating a complete booking flow.	Integrate the sandbox early, isolate payment logic behind a dedicated adapter, and test success, failure, cancellation, and callback scenarios. If the sandbox is unavailable, use a clearly labelled simulated payment flow for the academic demonstration and document the limitation; do not represent simulated payment as a real transaction.
Medium	**Scope growth.** Additional requests beyond the 14 core capabilities (for	Prioritize the 14 core capabilities, assess each change for schedule and effort impact, and defer optional features when necessary. The AI assistant and PWA

	example extra report types, new notification channels, AI, or PWA support) may delay core functionality or final testing.	will not be allowed to delay core-scope delivery.
Medium	**Limited team availability and testing time.** Academic commitments or uneven workload may reduce test coverage and deployment readiness.	Review workload weekly, assign owners and backup support for critical tasks, automate repeatable tests where practical, and reserve Sprint 5 for stabilization and documentation. If the schedule becomes constrained, move lower-priority core capabilities from Sprint 4 to Sprint 5 (no later than the Release Candidate on 05/Dec/26) and defer optional features.
Medium	**Integration defects between frontend, backend, and database.** Inconsistent API contracts or data assumptions may cause failures across connected components.	Agree on API contracts and data conventions before integration, conduct early integration tests, and use shared test data. Record interface changes and verify affected workflows after changes are merged.
Medium	**Incomplete or incorrect test data.** Missing or inconsistent movie, showtime, seat, or booking data may make demonstrations unreliable.	Prepare a controlled dataset and repeatable setup instructions. Validate test data before sprint reviews and maintain seed scripts or documented setup steps so the demonstration environment can be restored.

The Team Leader will review high-priority risks at every sprint review. If a risk threatens a milestone or the final academic demonstration, the team will record the issue, reassess the schedule, agree on corrective actions, and notify the project supervisor when appropriate.

4.3.4 Configuration Management

Configuration management will be used to control changes to source code, requirements, design documents, database scripts, tests, and release artifacts. A shared Git repository will serve as the primary version-controlled repository. An issue tracker or shared task board will record defects, change requests, task status, and decisions.

Version Control and Baselines

The main branch will contain the reviewed and stable project baseline. Development work will be performed in feature branches where practical. Changes will be merged only after the relevant review and checks have been completed. The team will tag demonstrable sprint builds as 0.1, 0.2, 0.3, 0.4, 0.9 (Release Candidate), and 1.0 so that each review point can be associated with a specific code baseline.

Configuration Items

The following artifacts will be managed as configuration items:

- Source code and application configuration templates.
- Database schema, migration scripts, and seed data.
- API contracts and interface documentation.

- UI assets and design documents.
- Requirements, backlog snapshots, and change records.
- Test cases, test scripts, test data, and test results.
- Build and deployment instructions.
- Sprint reports, release notes, and final project documentation.
- Demonstration instructions and supporting materials.

Secrets, passwords, access tokens, and private credentials must not be committed to the repository. Environment-specific values should be stored separately and documented using safe configuration examples.

Change and Problem Control

1. A defect or proposed change is recorded in the issue tracker with a description, priority, and relevant evidence.
2. The Team Leader and affected owners review its impact on requirements, architecture, interfaces, effort, schedule, and testing.
3. The team decides whether to fix, accept, defer, or reject the item and records the decision.
4. Approved work is assigned to a sprint and implemented in a controlled branch.
5. The change is reviewed and tested before merging.
6. Requirements, documentation, and release notes are updated when affected.
7. The team verifies the change in the relevant build and retains the resulting baseline.

Changes that affect the agreed scope or academic deliverables will be referred to the project supervisor when approval is required.

Naming, Versioning, and Retention

Files and documents will use clear, consistent names that identify the artifact and, where appropriate, its version or date. Release tags will identify the planned project builds. Final approved documents and release artifacts will be stored in the shared project workspace with access available to the team.

The Git repository and shared workspace will be the primary online storage locations. Important deliverables will be backed up through the available repository or workspace backup mechanisms, with a separate copy retained when practical. The team will verify that the final release tag, documentation, and necessary setup instructions can be retrieved before close-out.

If a change introduces a serious defect, the team will identify the last known stable commit or release tag and use it to restore a working baseline or create a corrective branch. Recovery actions and any resulting changes will be documented. Project materials will be retained in the shared workspace through final evaluation and any additional retention period required by the course or institution.

4.4 Project Close-out

Project close-out follows the final demonstration and completes the project after the three development phases (Inception, Elaboration, and Construction). At the end of the project, during Sprint 5, the team will verify that the 14 core capabilities have been reviewed against the agreed requirements and acceptance criteria. The team will summarize the final testing results, confirm that critical defects have been resolved or documented, and record any incomplete requirements and known limitations.

The team will prepare the final build and demonstration environment, archive the final Vision Document, SDP, backlog snapshot, architecture and database documentation, API contracts, test cases and results, release notes, and demonstration materials in the shared project workspace. The final demonstration will be presented to the project supervisor.

After the final demonstration on 12/Dec/26, the team will conduct a retrospective to review task allocation, communication, delays, and lessons learned. A final close-out summary will record the delivered capabilities, outstanding limitations, testing status, schedule deviations, and expenses if applicable. Each member will confirm that assigned work and documentation have been handed over, and any remaining responsibilities will be agreed upon by the team and supervisor.

The project will be considered closed when the final demonstration is complete, the deliverables and documentation are archived, and outstanding issues and lessons learned are documented. The retrospective and the final close-out summary are completed by 12/Dec/26.